Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 15 (2)

Roll No.:324

Name:Aditya Chauhan

Continued..

10) Calculate area of a ractangle using object as an argument to a method.

In [1]:
```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

class Demo:
    def show_area(self, obj):
```

```
        print("Area of Rectangle =", obj.area())

# Input
l = float(input("Enter length: "))
w = float(input("Enter width: "))

r = Rectangle(l, w)
d = Demo()
d.show_area(r)
```

```
Area of Rectangle = 100.0
```

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

In [2]:
```
class Square:
    def __init__(self, side):
        self.side = side
        self.a = 0

    def area(self):
        self.a = self.side * self.side
        self.output()

    def output(self):
        print("Area of Square =", self.a)

# Input
side = float(input("Enter side of square: "))
s = Square(side)
s.area()
```

```
Area of Square = 100.0
```

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

Also define a class method that compares the two sides of reactangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : THIS IS SQUARE.

In [3]:
```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
        self.a = 0

    @classmethod
    def check_shape(cls, length, width):
        if length == width:
            print("THIS IS SQUARE")
            return None
        else:
            return cls(length, width)

    def area(self):
        self.a = self.length * self.width
        self.output()

    def output(self):
        print("Area of Rectangle =", self.a)

# Input
l = float(input("Enter length: "))
w = float(input("Enter width: "))

r = Rectangle.check_shape(l, w)

if r is not None:
    r.area()
```

```
Area of Rectangle = 50.0
```

13) Define a class Square having a private attribute "side".

Implement get_side and set_side methods to accees the private attribute from outside of the class.

In [6]:
```
class Square:
    def __init__(self, side):
        self.__side = side   # private attribute

    def get_side(self):
        return self.__side

    def set_side(self, side):
        self.__side = side

# Create object
s = Square(5)

# Get value
print("Side =", s.get_side())

# Set new value
new_side = float(input("Enter new side: "))
s.set_side(new_side)

# Get updated value
print("Updated Side =", s.get_side())
```

```
Side = 5
Updated Side = 10.0
```

14) Create a class Profit that has a method named getProfit that accepts profit from the user.

Create a class Loss that has a method named getLoss that accepts loss from the user.

Create a class BalanceSheet that inherits from both classes Profit and Loss and calculates the balanace. It has two methods getBalance() and printBalance().

In [7]:
```
class Profit:
    def getProfit(self):
```

```
        self.profit = float(input("Enter profit: "))

class Loss:
    def getLoss(self):
        self.loss = float(input("Enter loss: "))

class BalanceSheet(Profit, Loss):
    def getBalance(self):
        self.balance = self.profit - self.loss

    def printBalance(self):
        print("Balance =", self.balance)

# Object creation
b = BalanceSheet()
b.getProfit()
b.getLoss()
b.getBalance()
b.printBalance()
```

```
Balance = 90.0
```

15) WAP to demonstrate all types of inheritance.

In [11]:

```
print("Single Inheritance")

class Parent:
    def show_parent(self):
        print("This is Parent class")

class Child(Parent):
    def show_child(self):
        print("This is Child class")

c = Child()
c.show_parent()
c.show_child()

print("\nMultilevel Inheritance")

class GrandParent:
```

```python
    def show_grandparent(self):
        print("This is GrandParent class")

class Parent(GrandParent):
    def show_parent(self):
        print("This is Parent class")

class Child(Parent):
    def show_child(self):
        print("This is Child class")


print("\nHierarchical Inheritance")

class Parent:
    def show_parent(self):
        print("This is Parent class")

class Child1(Parent):
    def show_child1(self):
        print("This is Child1 class")

class Child2(Parent):
    def show_child2(self):
        print("This is Child2 class")

c1 = Child1()
c2 = Child2()

c1.show_parent()
c1.show_child1()

c2.show_parent()
c2.show_child2()

print("\nHybrid Inheritance")

class A:
    def show_a(self):
        print("Class A")

class B(A):
```

```python
    def show_b(self):
        print("Class B")

class C(A):
    def show_c(self):
        print("Class C")

class D(B, C):
    def show_d(self):
        print("Class D")

d = D()
d.show_a()
d.show_b()
d.show_c()
d.show_d()
c = Child()
c.show_grandparent()
c.show_parent()
c.show_child()

print("\nMultiple Inheritance")

class Father:
    def show_father(self):
        print("This is Father class")

class Mother:
    def show_mother(self):
        print("This is Mother class")

class Child(Father, Mother):
    def show_child(self):
        print("This is Child class")

c = Child()
c.show_father()
c.show_mother()
c.show_child()

print("\nHierarchical Inheritance")
```

```python
class Parent:
    def show_parent(self):
        print("This is Parent class")

class Child1(Parent):
    def show_child1(self):
        print("This is Child1 class")

class Child2(Parent):
    def show_child2(self):
        print("This is Child2 class")

c1 = Child1()
c2 = Child2()

c1.show_parent()
c1.show_child1()

c2.show_parent()
c2.show_child2()

print("\nHybrid Inheritance")

class A:
    def show_a(self):
        print("Class A")

class B(A):
    def show_b(self):
        print("Class B")

class C(A):
    def show_c(self):
        print("Class C")

class D(B, C):
    def show_d(self):
        print("Class D")

d = D()
d.show_a()
d.show_b()
```

```python
d.show_c()
d.show_d()
```

```
Single Inheritance
This is Parent class
This is Child class

Multilevel Inheritance

Hierarchical Inheritance
This is Parent class
This is Child1 class
This is Parent class
This is Child2 class

Hybrid Inheritance
Class A
Class B
Class C
Class D
This is GrandParent class
This is Parent class
This is Child class

Multiple Inheritance
This is Father class
This is Mother class
This is Child class

Hierarchical Inheritance
This is Parent class
This is Child1 class
This is Parent class
This is Child2 class

Hybrid Inheritance
Class A
Class B
Class C
Class D
```

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class Employee that inherits from Person and adds a new attribute salary.

Override the **init** method in Employee to call the parent class's **init** method using the super() and then initialize the salary attribute.

In [12]:
```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Employee(Person):
    def __init__(self, name, age, salary):
        super().__init__(name, age)
        self.salary = salary

    def display(self):
        print("Name =", self.name)
        print("Age =", self.age)
        print("Salary =", self.salary)

# Input
name = input("Enter name: ")
age = int(input("Enter age: "))
salary = float(input("Enter salary: "))

e = Employee(name, age, salary)
e.display()
```

```
Name = Aditya
Age = 19
Salary = 250000.0
```

17) Create a Shape class with a draw method that is not implemented.

Create three child classes Rectangle, Circle, and Triangle that implement the draw method with their respective drawing behaviors.

Create a list of Shape objects that includes one instance of each child class, and then iterate through the list and call the draw method on each object.

In [13]:
```
class Shape:
    def draw(self):
        raise NotImplementedError("Subclass must implement draw method")

class Rectangle(Shape):
    def draw(self):
        print("Drawing a Rectangle")

class Circle(Shape):
    def draw(self):
        print("Drawing a Circle")

class Triangle(Shape):
    def draw(self):
        print("Drawing a Triangle")

# List of Shape objects
shapes = [Rectangle(), Circle(), Triangle()]

# Iterate and call draw()
for shape in shapes:
    shape.draw()
```

```
Drawing a Rectangle
Drawing a Circle
Drawing a Triangle
```

In [ ]: